claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-af646f28c19287962.jsonl`
- SHA-256: `70fdaea0d2debeddb5e6f64292cbb5a8b30bdc300f3c230ffefac5b0ce63e8ac`
- Source modified: `2026-07-03T17:36:07+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-03T17:34:18.101Z)

You are a code-review finder agent. Review target: PR #452 of badminton-highlight-indexer at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, branch codex/a2-golden-manifest-portability (HEAD 5a91701). Diff: git diff origin/master..HEAD. Context: this implements work item A2 from docs/ALPHA_LAUNCH_READINESS.md §7 ? "Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that fails loudly when labels or trajectories are missing."

ANGLE ? altitude. Is the fix at the right depth, or a band-aid on shared infrastructure?
1. The resolver hardcodes the owner's workspace layout (root.parent / "Annotation Setup" / "Trajectories|Golden Labelled", "Collect" legacy variant) inside backend/eval ? is baking one machine's folder names into repo code the right depth vs (a) writing the manifest with RELATIVE paths at ingest time (fix the producer, not every consumer), or (b) a checked-in path-mapping config? Read docs/data_pipeline/GOLDEN_VIDEOS.md ingestion flow and the ALPHA_LAUNCH_READINESS A2 row + §9 Q4 ("Should the portable corpus manifest live as a generated artifact, a checked-in redacted manifest, or both?") ? does this PR pre-empt or contradict the open question Q4? Is rebase-on-every-load the right mechanism vs rebase-once + --write (which the CLI already supports) and pointing tools at the resolved artifact?
2. Basename-only fallback matching: is silently matching by basename across search roots too permissive an altitude for a CORPUS-INTEGRITY tool (a wrong same-named file would be silently accepted as golden data)? Should a rebased hit be validated (row-count/hash/fps sanity) or at least logged? Check whether the resolver logs/reports WHICH paths were rebased (the CLI prints a count; do the library consumers get any signal?).
3. The manifest itself stays gitignored owner-side state ? does A2 "portable/reproducible" arguably require the manifest (or a redacted form) to be generated/checked-in, and does this PR document why it chose not to?
4. Is backend/eval the right home vs backend/utils or data_pipeline/ (who else will need path resolution ? collector reconciliation A3, feature regen A7)?
Only flag concrete altitude problems with the deeper alternative named. Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-03T17:34:20.005Z)

I'll analyze this PR for altitude issues by examining the diff, the manifest implementation, and the surrounding context.

3. user (2026-07-03T17:34:20.994Z)

```
 backend/eval/calibrate_local.py        |   5 +-
 backend/eval/distill_local.py          |   5 +-
 backend/eval/fusion_golden.py          |   4 +-
 backend/eval/golden_manifest.py        | 378 ++++++++++++++++++++++++++++++++++
 backend/eval/rally_seg_eval.py         |   4 +-
 backend/eval/rally_seq_proto.py        |   5 +-
 backend/eval/serve_contrast.py         |   4 +-
```

```
 backend/eval/served_gate_a.py          |   5 +-
 docs/ALPHA_LAUNCH_READINESS.md         |  11 +-
 docs/data_pipeline/GOLDEN_VIDEOS.md    |   8 +-
 tests/test_fusion_compare.py           |   4 +
 tests/test_fusion_golden_telemetry.py  |   3 +
 tests/test_golden_manifest.py          | 158 +++++++++++++
 tests/test_served_gate_a_regression.py |  21 +-
 training/gen0/harness.py               |   4 +-
 15 files changed, 586 insertions(+), 33 deletions(-)
```

4. assistant (2026-07-03T17:34:22.863Z)

Now let me read the key files to understand the context:

5. user (2026-07-03T17:34:23.821Z)

```
1       # Alpha Launch Readiness Plan
2
3       > **Status:** `DRAFT` - cross-repo execution tracker; **Owner:** @avidullu; **Created:**
2026-07-03; **Last updated:** 2026-07-03
4       > **Lifecycle:** `DRAFT -> IN PROGRESS -> DONE -> archived`
5       > **Tracking anchors:** Section 7 is the progress tracker and source of truth; indexed
in `docs/README.md` and `docs/DOC_STATUS.md`.
6       > **Relation to existing docs:** consolidates the launch-relevant pending items from
`NEXT_STEPS.md`, `ROADMAP.md`, `docs/data_pipeline/GOLDEN_VIDEOS.md`,
`R2_EVAL_METRIC_DESIGN.md`, `COMPUTE_DECOUPLED_SERVING/`, and the sibling `khelsutra` Studio
docs.
7       > **Honesty note:** `[verified]` means checked against the local synced repos on
2026-07-03. `[analysis]` means derived from a local command run on that snapshot. This is not
legal, financial, or launch approval.
8
9       ---
10
11      ## 0. TL;DR
12
13      The corpus is now large enough to move from "grow labels first" into alpha-readiness
work, but the 15-video headline is not itself a launch gate pass. `[verified]` The golden
manifest has 15 badminton videos; `[analysis]` the local manifest needed path rebasing before
manifest-driven commands ran; `[analysis]` only 6 of 15 feature JSONs had the full fusion
schema; `[analysis]` the n=15 quality baseline is stale and must be intentionally refreshed
before it can gate anything.
14
15      The first controlled alpha should be a hosted, authenticated product loop: upload a
video, process it, select rallies, compile, and download a reel. The owned-model/commercial
rally-detection claim remains a separate NO-GO until the ship-gate target is ratified and the
WASB checkpoint provenance is resolved.
16
17      ## 1. Problem & goal
18
19      The previous launch blockers were spread across roadmap docs and mixed together three
different questions:
20
21      - **Can an invited tester use the product without SSH?**
22      - **Can the 15-video golden corpus now support honest quality gates?**
23      - **Can we claim or ship an owned/commercial rally detector?**
24
25      This doc separates those into executable workstreams. The goal is to reach an invite-
only alpha where the product is usable end-to-end, the quality claims are honest and
reproducible, and the remaining model/legal gates are explicit rather than accidentally implied
by launch activity.
26
27      ## 2. Decisions locked
28
29      | # | Decision | Source / date | Implication |
30      |---|---|---|---|
```

D1	**Product alpha and owned-model launch are separate decisions.**	`NEXT_STEPS.md` NO-GO banner, 2026-06-30; verified 2026-07-03	We can launch a controlled hosted product loop without claiming the owned model is commercially cleared.
D2	**First alpha must be page-driven, not SSH/CLI-driven.**	`NEXT_STEPS.md` P0 UI-driven/shareable item; verified 2026-07-03	Hosted service, edge auth, upload, job polling, compile, and download are launch-critical.
D3	**The 15-video corpus is now a measurement asset, not a free launch pass.**	`[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md`; `[analysis]` local eval runs, 2026-07-03	Rebase/portability, feature completeness, baseline refresh, and gate ratification come before quality claims.
D4	**Do not promote served Gate A by default from the current n=15 evidence.**	`[analysis]` `backend.eval.serve_contrast` run, 2026-07-03	Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever until owner ratification.
D5	**P10/P11 corpus promotion and train/re-eval are alpha-plus unless owner reopens D13.**	sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`; verified 2026-07-03	First alpha can use manual/offline corpus promotion; automated flywheel follows after serving and gates are stable.

3. Verified snapshot, 2026-07-03

All five Git checkouts were fast-forwarded before analysis:

Repo	Branch	Synced HEAD
`badminton-highlight-indexer`	`master`	`e4914d1`
`khelsutra`	`master`	`b0f70a5`
`rally-annotator`	`main`	`ef9e088`
`sports-data-collector`	`master`	`bfc014f`
`sports-obsreport`	`main`	`a0c7252`

Corpus/eval facts:

- `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md` says `output/human_lovo_manifest.json` is the ingested golden corpus at `n = 15`.
- `[analysis]` The local manifest had stale absolute paths. A temporary rebased manifest found all 15 trajectories and labels in this workspace.
- `[analysis]` There are 15 `output/*_golden_features.json` files, but only 6 carry full shuttle/person/audio candidate features. The newer 9 are windowing-only schema and are skipped by fusion/person eval.
- `[analysis]` `python -m backend.eval.fusion_compare --features-dir output` loaded only 6 videos and measured person-fusion `Delta F1 = -0.021`; not promotable.
- `[analysis]` `python -m backend.eval.serve_contrast --manifest <rebased-n15>` measured proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.
- `[analysis]` `python -m training.gen0.harness --manifest <rebased-n15> --floor eval_baselines/heuristic_lovo_n6.json --version analysis-n15 --out <temp-out>` produced learned LOVO mean F1 `0.551`, `ship=False`; blockers remain stale n=6 floor/sign test, uncalibrated ship target, and WASB C3 provenance.
- `[analysis]` `python scripts/nightly_regression.py --manifest <rebased-n15> --features-dir output --no-report` reported default/milestone tolF1 `0.3636` vs baseline-off tolF1 `0.1382`, but marked the frozen baseline stale due to config/corpus changes.
- `[analysis]` `python -m backend.eval.ablation --features-dir output --granularity group` currently fails at import due to a circular import between `backend/eval/ablation.py` and `backend/eval/ablation_pdf.py`.

4. Launch strategy

4.1 Alpha lane: make the product usable

The alpha lane is successful when an invited tester can open the app, authenticate, upload a video, watch the job progress, pick rallies, compile a reel, and download it. This lane should avoid model-quality promises beyond the measured state and should default to the most reliable serving path.

4.2 Quality lane: make the 15-video corpus gate-ready

The quality lane is successful when the 15-video corpus is portable, reproducible, feature-complete, and backed by refreshed baselines. Only then should R2 tolF1, Gen-0/TemporalMaxer, or other model/promotion decisions be ratified.

4.3 Flywheel lane: make tester feedback useful

The flywheel lane is successful when user corrections can flow into the collector/vault path with consent/provenance and a visible re-eval delta. This is valuable for alpha learning, but it should not block the first controlled alpha if manual promotion can bridge the gap.

5. Risks and guardrails

Risk	Why it matters	Guardrail
Launch activity implies model launch approval	The engine docs explicitly say rally-detection product launch is NO-GO until ship target and WASB C3 clear	Keep all alpha copy scoped to invite-only processing, not owned-model claims
Stale local manifests create false confidence	Manifest-driven tools skipped or failed until paths were rebased	Make the corpus manifest portable and add a path/fixture smoke check
15-video corpus is only partially feature-complete	Fusion/person/audio eval is still n=6 today	Regenerate or upconvert full-fusion features for all 15 before fusion decisions
Baseline refresh accidentally rubber-stamps a regression	Nightly runner correctly reported stale baselines	Refresh baselines only after reviewer sign-off and record the command/output
P10/P11 expands scope before hosted alpha is usable	Corpus flywheel is valuable but can consume the alpha launch window	Keep P10/P11 behind explicit owner reopen unless alpha lane is already stable
WASB checkpoint provenance remains unresolved	Blocks commercial sharing of a trained model regardless of F1	Treat C3 as a hard model-release gate; do not hide it inside product launch tasks

6. Honest limits

Completing this doc's alpha lane does **not** prove the owned model is ready, that WASB weights are commercially clean, that the product can serve unbounded public traffic, that multi-sport is ready, or that every tester correction automatically improves the model.

Completing the quality lane does **not** by itself deploy a service. It only makes the measurement surface honest enough for a launch decision.

7. Deliverables & progress tracker

Legend: TODO = not started, IN PROGRESS = actively being worked, DONE = shipped/verified, GATED = blocked on owner/legal/platform decision. One PR per row unless the row explicitly names a cross-repo batch.

ID	Deliverable	Repo / owner	Depends on	Gated?	Status	PR
A0	Create this alpha readiness tracker and index it in docs	`badminton-highlight-indexer`	-	No	DONE	#451
A1	Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth ON, upload -> process -> jobs -> compile -> download verified	`khelsutra` + `badminton-highlight-indexer`	A0	Owner spend / CF config	TODO	-
A2	Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that fails loudly when labels or trajectories are missing	`badminton-highlight-indexer`	A0	No	IN PROGRESS	this PR
A3	Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile collector's six-video source manifest with the 15-video eval corpus	`sports-data-collector` + vault	A2	Owner upload/storage scope	TODO	-
A4	Refresh the n=15 nightly regression baseline intentionally and record the exact command/output; keep stale-baseline warnings until reviewed	`badminton-highlight-indexer`	A2	Reviewer sign-off	TODO	-
A5	Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for future Gen-0 comparisons	`badminton-highlight-indexer`	A2	No	TODO	-

A6	Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation on the n=15 corpus	`badminton-highlight-indexer`	A2	No	TODO	-
A7	Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so fusion/person/audio analyses run over all 15	`badminton-highlight-indexer`	A2	GPU/data availability	TODO	-
A8	Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly promote/reject person fusion, Gate A served-default, and any other R-series default flips	`badminton-highlight-indexer`	A6, A7	Reviewer sign-off	TODO	-
A9	Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-serving artifact bundle and gate verdict	`badminton-highlight-indexer`	A5, A8	Owner approval for M0.4 scope	TODO	-
A10	Decide and record the alpha ship-gate target: metric, corpus slice, minimum threshold, significance rule, and what claims are allowed	Owner + `badminton-highlight-indexer`	A4, A5, A8	Owner decision	GATED	-
A11	Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred, document "no commercial model sharing" copy guardrail	Owner/legal	A10	Legal/provenance	GATED	-
A12	Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion, `train`/`reeval` job, delta-F1 surface, and alignment with `COMPUTE_DECOUPLED_SERVING` M11	`khelsutra` + `badminton-highlight-indexer` + collector	A1, A3	Owner D13 reopen (`khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`)	GATED	-
A13	Alpha launch review: runbook, privacy/consent copy, observability/job-failure triage, known limits, rollback/disable steps, and owner GO/NO-GO	Cross-repo	A1, A4, A10, A11	Owner launch decision	TODO	-

8. Concrete next PR sequence

1. **Docs PR:** land this tracker.
2. **Corpus portability PR:** rebase/portable manifest tooling plus missing-file smoke test.
3. **Hosted alpha PR/batch:** health/auth/upload/process/job/compile/download verification and runbook updates.
4. **Baseline PR:** intentional n=15 nightly baseline refresh and heuristic floor refresh.
5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun promotion/rejection decisions.
6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy limits, and launch checklist.

A2 smoke command:

```powershell
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15 --write output/human_lovo_manifest.local.json
```

The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.

9. Open questions

1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should P10/P11 be reopened before any tester wave?
2. **Owner/platform:** Which hosted compute target is the next stand-up: Cloud Run GPU, GCE L4 VM, or another approved GPU host?
3. **Owner/legal:** What is the acceptable alpha wording while WASB C3 remains unresolved?
4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a checked-in redacted manifest, or both?
5. **Engineering:** Should n=15 feature regeneration target full fusion schema for every clip, or should the eval loaders support mixed schemas with explicit capability tags?

```
138     ## 10. Definition of done
139
140     - [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without
SSH.
141     - [ ] Alpha service is gated behind the chosen auth posture and has a documented
disable/rollback path.
142     - [ ] n=15 corpus is portable enough that manifest-driven quality commands do not rely
on stale local absolute paths.
143     - [ ] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
144     - [ ] R2/tolF1 gate is promoted or rejected with a recorded ablation result.
145     - [ ] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected
on the 15-video corpus.
146     - [ ] Ship-gate target and allowed alpha claims are recorded.
147     - [ ] WASB C3 status is recorded and reflected in alpha messaging.
148     - [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for
alpha-plus.
149     - [ ] Owner records GO/NO-GO for the first alpha wave.
150
151     ## 11. References
152
153     Internal docs:
154
155     - `docs/NEXT_STEPS.md`
156     - `docs/ROADMAP.md`
157     - `docs/data_pipeline/GOLDEN_VIDEOS.md`
158     - `docs/R2_EVAL_METRIC_DESIGN.md`
159     - `docs/RALLY_DETECTION_FIXES_PLAN.md`
160     - `docs/COMPUTE_DECOUPLED_SERVING/README.md`
161     - sibling `khelsutra/NEXT_STEPS.md`
162     - sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`
163     - sibling `khelsutra/docs/PER_RALLY_SELECTION.md`
164     - sibling `sports-data-collector/data/golden_source_videos.json`
165
166     Local verification commands used for the 2026-07-03 snapshot:
167
168     ```powershell
169     python -m backend.eval.fusion_compare --features-dir output
170     python -m backend.eval.serve_contrast --manifest <rebased-n15-manifest>
171     python -m training.gen0.harness --manifest <rebased-n15-manifest> --floor
eval_baselines\heuristic_lovo_n6.json --version analysis-n15 --out <temp-out>
172     python scripts\nightly_regression.py --manifest <rebased-n15-manifest> --features-dir
output --no-report
173     python -m backend.eval.ablation --features-dir output --granularity group
174     ```
175
176     ---
177
178     ### Changelog
179
180     - 2026-07-03 - Created from a synced five-repo review of `NEXT_STEPS`, `ROADMAP`,
golden-corpus docs, local eval artifacts, and the current 15-video corpus state.
181
```

6. user (2026-07-03T17:34:24.024Z)

```
1       # Golden-labeled video registry
2
3       **Purpose.** The single source of truth for every **golden** (human rally-boundary-
labeled) video ? what
4       it is, where the source proxy lives, how many rallies, and whether it's ingested into
the eval corpus.
5       Use this to **isolate the golden videos into a dedicated location** and to keep the
growing golden set
6       auditable. Updated whenever a new video is labeled + ingested.
7
```

> **This is the GROWING GOLDEN-finalized set** in the dual-eval-set discipline (see
> [QUALITY_ITERATIONS.md](../QUALITY_ITERATIONS.md), the 2026-06-16 capstone + `eval-dual-set-regression`):
> a **RAW eval set** (currently-bad videos) + this **golden-finalized eval set** (promoted when at-par) +
> a separate **golden TRAINING set**. Every quality launch must not regress on either eval set.
> Privacy-safe, video-free *shipping* of these = [GOLDEN_REGRESSION_FIXTURES.md](../GOLDEN_REGRESSION_FIXTURES.md).

Contracts / conventions

- **Labels:** rally-annotator CSV (`rally_number,start_time,end_time,ending_reason,sport,shots_count`, decimal seconds) ? ingested by `backend/eval/gt_loader.py` (reads `start_time`/`end_time`).
- **Alignment is mandatory before trusting any score.** The label timebase MUST match the windowed proxy (same fps + frame count). Verify with `ffprobe` (the `largetest_doubles` near-miss is why) ? ideally annotate the *exact* proxy the detector ran on (then `video_id` = MD5 matches and alignment is free).
- **`video_id`** = MD5 of the proxy (`backend/utils/hashing.py::compute_video_id`); it keys the WASB trajectory cache, the Gemini reference rows, and the golden features.
- **Ingestion** = copy the trajectory + labels to durable paths, write `output/<name>_golden_features.json` (`gts` + candidates), append a row to `output/human_lovo_manifest.json`. (`scratch/at_par.py` scores a single video vs Gemini + golden; `backend/eval/rally_seg_eval.py` runs the whole corpus.)
- **Path smoke check:** before running corpus evals, run `python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15`. The checker rebases stale absolute paths against sibling `Annotation Setup` folders and `output/`, and exits non-zero if any required trajectory or labels file is missing. Add `--write output/human_lovo_manifest.local.json` when a resolved local manifest artifact is useful.
- **Guardrails:** owned footage only; **minors excluded**; labels never derived from a paid LLM (Gemini is inference/reference only). Commercial-clean.

Ingested golden corpus ? `output/human_lovo_manifest.json` (n = 15)

#	name	sport	rallies	fps	source proxy	video_id	labeled
1	adarsh_avi_singles	badminton (S)	96	59.94	(original 6)	?	pre-06-16
2	kushagra_singles	badminton (S)	32	59.94	(original 6)	?	pre-06-16
3	largetest_doubles	badminton (D)	49	29.97	(original 6)	?	pre-06-16
4	gbaaddy	badminton	48	59.94	(original 6)	?	pre-06-16
5	testlarge_short	badminton	6	29.97	(original 6)	?	pre-06-16
6	mahadevpura_singles	badminton (S)	31	29.97	(original 6)	?	pre-06-16
7	mahadevpura_2	badminton (multicourt)	40	59.94	`?/Roora Request - June 15/[Done] Video 3 ?/mahadevpura-2_proxy.mp4`	`038e0e0915a2`	2026-06-16
8	GX010128	badminton (multicourt)	52	59.94	`?/Golden Label Videos Request - June 15/[Done] Video 13 ?/GX010128.MP4`	`db84f3e65318`	2026-06-16
9	mahadevpura_1	badminton (S, continuous)	10	59.94	`?/[Done] Video 14 - Badminton/mahadevpura-1.MP4`	`38bef75d78cb`	2026-06-17

Labeled, ingestion in progress (WASB trajectory extracting)

name	sport	rallies	fps	source proxy	status
GX030094	badminton	41	**29.97**	`?/Golden Label ? /[Done] Video 6 - Badminton/GX030094_proxy.mp4`	**ingested** (in `human_lovo_manifest.json`)

```
52      | Badminton BXH 2 | badminton (multicourt) | 44 | 59.94 | `?/[Done] Video 10 - Badminton
Multicourt/Badminton BXH 2_proxy.mp4` | **ingested** (in `human_lovo_manifest.json`) |
53
54      ## Source folder & not-yet-labeled
55      - **Annotation root:** `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos
Request - June 15\`
56        (+ the earlier `Roora Request - June 15\`). `[Done]` prefix in a dir name = golden-
labeled.
57      - **Labeled badminton dirs:** Video 3 (=mahadevpura-2), 6 (=GX030094), 10 (=BXH 2), 13
(=GX010128, a
58        re-label of #8), 14 (=mahadevpura-1). **Video 13 is a duplicate of #8** (already
ingested).
59      - **Not yet labeled / other sports:** Video 1, 4, 7, 9, 12 (badminton, pending); Video
2, 8 (table tennis);
60        Video 5, 11 (pickleball). Badminton is the current focus (alpha customers);
TT/pickleball labeled later.
61
62      ## How to isolate the golden videos
63      Each row's **source proxy** column is the file to copy into a dedicated golden-videos
location. The labels
64      live beside the proxy (`<name>.rallies.csv`) and a durable copy is in
`output/<name>.rallies.csv`; the
65      trajectory is in `Annotation Setup/Trajectories/` (older manifests may still say
`Collect/Trajectories/`;
66      use the path smoke checker above). Keep this table in sync on every new ingest.
67
```

7. assistant (2026-07-03T17:34:27.311Z)

Now let me examine the main implementation file:

8. user (2026-07-03T17:34:27.976Z)

```
1       """Portable golden-corpus manifest loader and smoke checker.
2
3       The owner-side golden manifest is intentionally gitignored and has historically
4       carried absolute paths from whichever box ingested a clip. This module keeps that
5       artifact usable after the workspace moves by resolving stale paths against the
6       current repo layout and failing loudly when required corpus files are absent.
7
8       Default local layout supported:
9         - ../Annotation Setup/Trajectories/<trajectory>.csv
10        - ../Annotation Setup/Golden Labelled/<labels>.csv
11        - output/<labels>.csv
12
13      Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14      """
15
16      from __future__ import annotations
17
18      import argparse
19      import json
20      import os
21      import sys
22      from dataclasses import dataclass
23      from pathlib import Path, PureWindowsPath
24      from typing import Any, Iterable, Sequence
25
26      DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27      SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28      DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30      _FIELD_DIR = {
31          "trajectory": "Trajectories",
32          "labels": "Golden Labelled",
```

```
    }


    @dataclass(frozen=True)
    class ManifestIssue:
        """One manifest path problem found by the resolver."""

        name: str
        field: str
        path: str
        reason: str

        def message(self) -> str:
            shown = f" {self.path!r}" if self.path else ""
            return f"{self.name}: {self.field}{shown} - {self.reason}"


    class ManifestPathError(RuntimeError):
        """Raised when a strict manifest load finds missing required files."""

        def __init__(self, manifest_path: str | os.PathLike[str], issues:
Sequence[ManifestIssue]):
            self.manifest_path = os.fspath(manifest_path)
            self.issues = list(issues)
            detail = "\n".join(f"  - {i.message()}" for i in self.issues[:12])
            more = "" if len(self.issues) <= 12 else f"\n  ... {len(self.issues) - 12} more"
            super().__init__(
                f"golden manifest path check failed for {self.manifest_path}: "
                f"{len(self.issues)} issue(s)\n{detail}{more}"
            )


    def repo_root() -> Path:
        """Return the repository root for this module."""

        return Path(__file__).resolve().parents[2]


    def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
        """Load the flat golden manifest list and validate its outer shape."""

        path = Path(manifest_path)
        with path.open(encoding="utf-8") as f:
            data = json.load(f)
        if not isinstance(data, list):
            raise ValueError(f"golden manifest must be a JSON list: {path}")
        out: list[dict[str, Any]] = []
        for i, row in enumerate(data):
            if not isinstance(row, dict):
                raise ValueError(f"golden manifest row {i} is not an object: {path}")
            out.append(dict(row))
        return out


    def resolve_manifest_paths(
        manifest_path: str | os.PathLike[str],
        *,
        repo_root_path: str | os.PathLike[str] | None = None,
        extra_roots: Iterable[str | os.PathLike[str]] = (),
        required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
    ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
        """Load and rebase required path fields in a golden manifest.

        Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
        metadata but replace any resolved ``trajectory`` / ``labels`` path with the
```

```
        existing local path. Missing required fields are reported in ``issues``.
        """

        manifest = load_manifest(manifest_path)
        return resolve_manifest_specs(
            manifest,
            manifest_path,
            repo_root_path=repo_root_path,
            extra_roots=extra_roots,
            required_fields=required_fields,
        )


    def load_resolved_manifest(
        manifest_path: str | os.PathLike[str],
        *,
        repo_root_path: str | os.PathLike[str] | None = None,
        extra_roots: Iterable[str | os.PathLike[str]] = (),
        required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
        strict: bool = True,
    ) -> list[dict[str, Any]]:
        """Load a manifest with stale paths rebased; optionally raise on missing files."""

        specs, issues = resolve_manifest_paths(
            manifest_path,
            repo_root_path=repo_root_path,
            extra_roots=extra_roots,
            required_fields=required_fields,
        )
        if strict and issues:
            raise ManifestPathError(manifest_path, issues)
        return specs


    def resolve_manifest_specs(
        specs: Sequence[dict[str, Any]],
        manifest_path: str | os.PathLike[str],
        *,
        repo_root_path: str | os.PathLike[str] | None = None,
        extra_roots: Iterable[str | os.PathLike[str]] = (),
        required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
    ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
        """Resolve an already-loaded manifest list."""

        manifest = Path(manifest_path)
        root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
        roots = [Path(p) for p in _env_roots()]
        roots.extend(Path(p) for p in extra_roots)
        required = tuple(required_fields)

        resolved: list[dict[str, Any]] = []
        issues: list[ManifestIssue] = []
        for idx, spec in enumerate(specs):
            row = dict(spec)
            name = str(row.get("name") or f"row[{idx}]")
            for field in required:
                raw = row.get(field)
                if not raw:
                    issues.append(ManifestIssue(name, field, "", "required field missing"))
                    continue
                hit = resolve_existing_path(
                    str(raw),
                    field,
                    manifest_path=manifest,
                    repo_root_path=root,
```

```
                    extra_roots=roots,
                )
                if hit is None:
                    issues.append(ManifestIssue(name, field, str(raw), "file not found"))
                else:
                    row[field] = str(hit)
            resolved.append(row)
        return resolved, issues


    def resolve_existing_path(
        raw_path: str,
        field: str,
        *,
        manifest_path: str | os.PathLike[str],
        repo_root_path: str | os.PathLike[str],
        extra_roots: Iterable[str | os.PathLike[str]] = (),
    ) -> Path | None:
        """Resolve one manifest path field to an existing file, if possible."""

        for candidate in _candidate_paths(
            raw_path,
            field,
            Path(manifest_path),
            Path(repo_root_path),
            [Path(p) for p in extra_roots],
        ):
            if candidate.is_file():
                return candidate.resolve()
        return None


    def _env_roots() -> list[Path]:
        raw = os.environ.get(SEARCH_ROOTS_ENV, "")
        if not raw:
            return []
        return [Path(p) for p in raw.split(os.pathsep) if p.strip()]


    def _basename(raw_path: str) -> str:
        # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
        win_name = PureWindowsPath(raw_path).name
        return win_name or Path(raw_path).name


    def _candidate_paths(
        raw_path: str,
        field: str,
        manifest_path: Path,
        root: Path,
        extra_roots: Sequence[Path],
    ) -> list[Path]:
        basename = _basename(raw_path)
        raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
        manifest_dir = manifest_path.resolve().parent
        candidates: list[Path] = [raw]

        if not raw.is_absolute():
            candidates.extend([manifest_dir / raw, root / raw])

        for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
            candidates.append(directory / basename)
        return _dedup_paths(candidates)


```

```
    def _candidate_dirs(
        field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
    ) -> list[Path]:
        field_dir = _FIELD_DIR.get(field)
        dirs: list[Path] = [manifest_dir]
        if field_dir:
            dirs.extend(
                [
                    root.parent / "Annotation Setup" / field_dir,
                    root.parent / "Annotation Setup" / "Collect" / field_dir,
                    root / "output",
                ]
            )
        dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
        return _dedup_paths(dirs)


    def _expand_root(root: Path, field: str) -> list[Path]:
        field_dir = _FIELD_DIR.get(field)
        dirs = [root, root / "output"]
        if field_dir:
            dirs.extend(
                [
                    root / field_dir,
                    root / "Annotation Setup" / field_dir,
                    root / "Annotation Setup" / "Collect" / field_dir,
                ]
            )
        return dirs


    def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
        out: list[Path] = []
        seen: set[str] = set()
        for path in paths:
            try:
                key = os.path.normcase(os.path.abspath(os.fspath(path)))
            except OSError:
                key = os.fspath(path)
            if key in seen:
                continue
            seen.add(key)
            out.append(path)
        return out


    def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
        """Write a resolved manifest JSON atomically enough for local tooling."""

        out = Path(path)
        out.parent.mkdir(parents=True, exist_ok=True)
        tmp = out.with_name(out.name + ".tmp")
        with tmp.open("w", encoding="utf-8") as f:
            json.dump(list(specs), f, indent=2)
            f.write("\n")
        os.replace(tmp, out)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            description="Check/rebase the local golden corpus manifest paths."
        )
        p.add_argument("--manifest", default=DEFAULT_MANIFEST)
        p.add_argument(
```

```
            "--repo-root",
            default=None,
            help="repo root used for default sibling output/Annotation Setup lookups",
        )
        p.add_argument(
            "--root",
            action="append",
            default=[],
            help=(
                "additional search root; repeatable. Also accepts roots via "
                f"{SEARCH_ROOTS_ENV} separated by os.pathsep."
            ),
        )
        p.add_argument(
            "--require",
            action="append",
            choices=sorted(_FIELD_DIR),
            default=None,
            help="path field to require; repeatable. Defaults to trajectory and labels.",
        )
        p.add_argument(
            "--min-count",
            type=int,
            default=None,
            help="fail unless at least this many manifest rows are present",
        )
        p.add_argument(
            "--write",
            default=None,
            help="optional path to write the resolved local manifest JSON",
        )
        return p


    def main(argv: Sequence[str] | None = None) -> int:
        args = build_parser().parse_args(argv)
        required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
        try:
            original = load_manifest(args.manifest)
            resolved, issues = resolve_manifest_specs(
                original,
                args.manifest,
                repo_root_path=args.repo_root,
                extra_roots=args.root,
                required_fields=required,
            )
        except (OSError, ValueError, json.JSONDecodeError) as e:
            print(f"ERROR: {e}", file=sys.stderr)
            return 2

        if args.min_count is not None and len(resolved) < args.min_count:
            issues.append(
                ManifestIssue(
                    "manifest",
                    "rows",
                    str(len(resolved)),
                    f"expected at least {args.min_count} row(s)",
                )
            )

        if issues:
            print(
                f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
                file=sys.stderr,
            )
```

```
356             for issue in issues:
357                 print(f"  - {issue.message()}", file=sys.stderr)
358             return 2
359 
360         rebased = sum(
361             1
362             for before, after in zip(original, resolved)
363             for field in required
364             if str(before.get(field, "")) != str(after.get(field, ""))
365         )
366         print(
367             f"OK: {args.manifest} has {len(resolved)} video(s); "
368             f"{len(resolved) * len(required)} required path(s) present; "
369             f"{rebased} path(s) rebased."
370         )
371         if args.write:
372             write_manifest(args.write, resolved)
373             print(f"wrote resolved manifest: {args.write}")
374         return 0
375 
376 
377     if __name__ == "__main__":
378         raise SystemExit(main())
379 
```

9. user (2026-07-03T17:34:29.154Z)

```
diff --git a/backend/eval/golden_manifest.py b/backend/eval/golden_manifest.py
new file mode 100644
index 0000000..7a01677
--- /dev/null
+++ b/backend/eval/golden_manifest.py
@@ -0,0 +1,378 @@
+"""Portable golden-corpus manifest loader and smoke checker.
+
+The owner-side golden manifest is intentionally gitignored and has historically
+carried absolute paths from whichever box ingested a clip. This module keeps that
+artifact usable after the workspace moves by resolving stale paths against the
+current repo layout and failing loudly when required corpus files are absent.
+
+Default local layout supported:
+  - ../Annotation Setup/Trajectories/<trajectory>.csv
+  - ../Annotation Setup/Golden Labelled/<labels>.csv
+  - output/<labels>.csv
+
+Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
+"""
+
+from __future__ import annotations
+
+import argparse
+import json
+import os
+import sys
+from dataclasses import dataclass
+from pathlib import Path, PureWindowsPath
+from typing import Any, Iterable, Sequence
+
+DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
+SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
+DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
+
+_FIELD_DIR = {
+    "trajectory": "Trajectories",
+    "labels": "Golden Labelled",
```

```
+}
+
+
+@dataclass(frozen=True)
+class ManifestIssue:
+    """One manifest path problem found by the resolver."""
+
+    name: str
+    field: str
+    path: str
+    reason: str
+
+    def message(self) -> str:
+        shown = f" {self.path!r}" if self.path else ""
+        return f"{self.name}: {self.field}{shown} - {self.reason}"
+
+
+class ManifestPathError(RuntimeError):
+    """Raised when a strict manifest load finds missing required files."""
+
+    def __init__(self, manifest_path: str | os.PathLike[str], issues: Sequence[ManifestIssue]):
+        self.manifest_path = os.fspath(manifest_path)
+        self.issues = list(issues)
+        detail = "\n".join(f"  - {i.message()}" for i in self.issues[:12])
+        more = "" if len(self.issues) <= 12 else f"\n  ... {len(self.issues) - 12} more"
+        super().__init__(
+            f"golden manifest path check failed for {self.manifest_path}: "
+            f"{len(self.issues)} issue(s)\n{detail}{more}"
+        )
+
+
+def repo_root() -> Path:
+    """Return the repository root for this module."""
+
+    return Path(__file__).resolve().parents[2]
+
+
+def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
+    """Load the flat golden manifest list and validate its outer shape."""
+
+    path = Path(manifest_path)
+    with path.open(encoding="utf-8") as f:
+        data = json.load(f)
+    if not isinstance(data, list):
+        raise ValueError(f"golden manifest must be a JSON list: {path}")
+    out: list[dict[str, Any]] = []
+    for i, row in enumerate(data):
+        if not isinstance(row, dict):
+            raise ValueError(f"golden manifest row {i} is not an object: {path}")
+        out.append(dict(row))
+    return out
+
+
+def resolve_manifest_paths(
+    manifest_path: str | os.PathLike[str],
+    *,
+    repo_root_path: str | os.PathLike[str] | None = None,
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
+) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
+    """Load and rebase required path fields in a golden manifest.
+
+    Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
+    metadata but replace any resolved ``trajectory`` / ``labels`` path with the
+    existing local path. Missing required fields are reported in ``issues``.
```

```
+    """
+
+    manifest = load_manifest(manifest_path)
+    return resolve_manifest_specs(
+        manifest,
+        manifest_path,
+        repo_root_path=repo_root_path,
+        extra_roots=extra_roots,
+        required_fields=required_fields,
+    )
+
+
+def load_resolved_manifest(
+    manifest_path: str | os.PathLike[str],
+    *,
+    repo_root_path: str | os.PathLike[str] | None = None,
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
+    strict: bool = True,
+) -> list[dict[str, Any]]:
+    """Load a manifest with stale paths rebased; optionally raise on missing files."""
+
+    specs, issues = resolve_manifest_paths(
+        manifest_path,
+        repo_root_path=repo_root_path,
+        extra_roots=extra_roots,
+        required_fields=required_fields,
+    )
+    if strict and issues:
+        raise ManifestPathError(manifest_path, issues)
+    return specs
+
+
+def resolve_manifest_specs(
+    specs: Sequence[dict[str, Any]],
+    manifest_path: str | os.PathLike[str],
+    *,
+    repo_root_path: str | os.PathLike[str] | None = None,
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
+) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
+    """Resolve an already-loaded manifest list."""
+
+    manifest = Path(manifest_path)
+    root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
+    roots = [Path(p) for p in _env_roots()]
+    roots.extend(Path(p) for p in extra_roots)
+    required = tuple(required_fields)
+
+    resolved: list[dict[str, Any]] = []
+    issues: list[ManifestIssue] = []
+    for idx, spec in enumerate(specs):
+        row = dict(spec)
+        name = str(row.get("name") or f"row[{idx}]")
+        for field in required:
+            raw = row.get(field)
+            if not raw:
+                issues.append(ManifestIssue(name, field, "", "required field missing"))
+                continue
+            hit = resolve_existing_path(
+                str(raw),
+                field,
+                manifest_path=manifest,
+                repo_root_path=root,
+                extra_roots=roots,
```

```
+            )
+            if hit is None:
+                issues.append(ManifestIssue(name, field, str(raw), "file not found"))
+            else:
+                row[field] = str(hit)
+        resolved.append(row)
+    return resolved, issues
+
+
+def resolve_existing_path(
+    raw_path: str,
+    field: str,
+    *,
+    manifest_path: str | os.PathLike[str],
+    repo_root_path: str | os.PathLike[str],
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+) -> Path | None:
+    """Resolve one manifest path field to an existing file, if possible."""
+
+    for candidate in _candidate_paths(
+        raw_path,
+        field,
+        Path(manifest_path),
+        Path(repo_root_path),
+        [Path(p) for p in extra_roots],
+    ):
+        if candidate.is_file():
+            return candidate.resolve()
+    return None
+
+
+def _env_roots() -> list[Path]:
```

10. assistant (2026-07-03T17:34:31.838Z)

Now let me examine how the manifest is used by consumers and check the integration points:

11. user (2026-07-03T17:34:33.669Z)

```
diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
 from __future__ import annotations

 import argparse
-import json
 from typing import Callable, Dict, List, Tuple

 from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
     leave_one_video_out,
     select_best,
 )
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.pipeline.detectors import rally_gate
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
     ap.add_argument("--iou", type=float, default=0.5)
     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
```

```
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, metric=args.metric)


diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--- a/backend/eval/distill_local.py
+++ b/backend/eval/distill_local.py
@@ -24,13 +24,13 @@ Run:
 from __future__ import annotations

 import argparse
-import json
 from typing import Dict, List, Optional, Tuple

 from backend.eval import windowing as _windowing
 from backend.eval.calibrate_wasb import load_golden_gts
 from backend.eval.classifier import LogisticRegression
 from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import score
 from backend.eval.training import label_candidates
 from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
     )
     ap.add_argument("--model-out", default="output/local_rally_model.json")
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)


diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index dd3f6a1..aee6d7e 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional

 from backend.eval import distill_local
 from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.training import label_candidates
 from backend.pipeline.detectors import fusion_features as ff
 from backend.pipeline.detectors import rally_gate
@@ -160,8 +161,7 @@ def run(
     fresh: bool = False,
     smallest_first: bool = False,
 ) -> List[str]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
     if smallest_first:
         # Process small videos first for quick feedback (the slow full match lands last);
         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
diff --git a/backend/eval/golden_manifest.py b/backend/eval/golden_manifest.py
new file mode 100644
index 0000000..7a01677
--- /dev/null
+++ b/backend/eval/golden_manifest.py
@@ -0,0 +1,378 @@
+"""Portable golden-corpus manifest loader and smoke checker.
+
+The owner-side golden manifest is intentionally gitignored and has historically
```

```
+carried absolute paths from whichever box ingested a clip. This module keeps that
+artifact usable after the workspace moves by resolving stale paths against the
+current repo layout and failing loudly when required corpus files are absent.
+
+Default local layout supported:
+  - ../Annotation Setup/Trajectories/<trajectory>.csv
+  - ../Annotation Setup/Golden Labelled/<labels>.csv
+  - output/<labels>.csv
+
+Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
+"""
+
+from __future__ import annotations
+
+import argparse
+import json
+import os
+import sys
+from dataclasses import dataclass
+from pathlib import Path, PureWindowsPath
+from typing import Any, Iterable, Sequence
+
+DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
+SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
+DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
+
+_FIELD_DIR = {
+    "trajectory": "Trajectories",
+    "labels": "Golden Labelled",
+}
+
+
+@dataclass(frozen=True)
+class ManifestIssue:
+    """One manifest path problem found by the resolver."""
+
+    name: str
+    field: str
+    path: str
+    reason: str
+
+    def message(self) -> str:
+        shown = f" {self.path!r}" if self.path else ""
+        return f"{self.name}: {self.field}{shown} - {self.reason}"
+
+
+class ManifestPathError(RuntimeError):
+    """Raised when a strict manifest load finds missing required files."""
+
+    def __init__(self, manifest_path: str | os.PathLike[str], issues: Sequence[ManifestIssue]):
+        self.manifest_path = os.fspath(manifest_path)
+        self.issues = list(issues)
+        detail = "\n".join(f"  - {i.message()}" for i in self.issues[:12])
+        more = "" if len(self.issues) <= 12 else f"\n  ... {len(self.issues) - 12} more"
+        super().__init__(
+            f"golden manifest path check failed for {self.manifest_path}: "
+            f"{len(self.issues)} issue(s)\n{detail}{more}"
+        )
+
+
+def repo_root() -> Path:
+    """Return the repository root for this module."""
+
+    return Path(__file__).resolve().parents[2]
+
```

```
+
+def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
+    """Load the flat golden manifest list and validate its outer shape."""
+
+    path = Path(manifest_path)
+    with path.open(encoding="utf-8") as f:
+        data = json.load(f)
+    if not isinstance(data, list):
+        raise ValueError(f"golden manifest must be a JSON list: {path}")
+    out: list[dict[str, Any]] = []
+    for i, row in enumerate(data):
+        if not isinstance(row, dict):
+            raise ValueError(f"golden manifest row {i} is not an object: {path}")
+        out.append(dict(row))
+    return out
+
+
+def resolve_manifest_paths(
+    manifest_path: str | os.PathLike[str],
+    *,
+    repo_root_path: str | os.PathLike[str] | None = None,
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
+) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
+    """Load and rebase required path fields in a golden manifest.
+
+    Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
+    metadata but replace any resolved ``trajectory`` / ``labels`` path with the
+    existing local path. Missing required fields are reported in ``issues``.
+    """
+
+    manifest = load_manifest(manifest_path)
+    return resolve_manifest_specs(
+        manifest,
+        manifest_path,
+        repo_root_path=repo_root_path,
+        extra_roots=extra_roots,
+        required_fields=required_fields,
+    )
+
+
+def load_resolved_manifest(
+    manifest_path: str | os.PathLike[str],
+    *,
+    repo_root_path: str | os.PathLike[str] | None = None,
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
+    strict: bool = True,
+) -> list[dict[str, Any]]:
+    """Load a manifest with stale paths rebased; optionally raise on missing files."""
+
+    specs, issues = resolve_manifest_paths(
+        manifest_path,
+        repo_root_path=repo_root_path,
+        extra_roots=extra_roots,
+        required_fields=required_fields,
+    )
+    if strict and issues:
+        raise ManifestPathError(manifest_path, issues)
+    return specs
+
+
+def resolve_manifest_specs(
+    specs: Sequence[dict[str, Any]],
+    manifest_path: str | os.PathLike[str],
```

```
+    *,
+    repo_root_path: str | os.PathLike[str] | None = None,
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
+) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
+    """Resolve an already-loaded manifest list."""
+
+    manifest = Path(manifest_path)
+    root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
+    roots = [Path(p) for p in _env_roots()]
+    roots.extend(Path(p) for p in extra_roots)
+    required = tuple(required_fields)
+
+    resolved: list[dict[str, Any]] = []
+    issues: list[ManifestIssue] = []
+    for idx, spec in enumerate(specs):
+        row = dict(spec)
+        name = str(row.get("name") or f"row[{idx}]")
+        for field in required:
+            raw = row.get(field)
+            if not raw:
+                issues.append(ManifestIssue(name, field, "", "required field missing"))
+                continue
+            hit = resolve_existing_path(
+                str(raw),
+                field,
+                manifest_path=manifest,
+                repo_root_path=root,
+                extra_roots=roots,
+            )
+            if hit is None:
+                issues.append(ManifestIssue(name, field, str(raw), "file not found"))
+            else:
+                row[field] = str(hit)
+        resolved.append(row)
+    return resolved, issues
+
+
+def resolve_existing_path(
+    raw_path: str,
+    field: str,
+    *,
+    manifest_path: str | os.PathLike[str],
+    repo_root_path: str | os.PathLike[str],
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+) -> Path | None:
+    """Resolve one manifest path field to an existing file, if possible."""
+
+    for candidate in _candidate_paths(
+        raw_path,
+        field,
+        Path(manifest_path),
+        Path(repo_root_path),
+        [Path(p) for p in extra_roots],
+    ):
+        if candidate.is_file():
+            return candidate.resolve()
+    return None
+
+
+def _env_roots() -> list[Path]:
+    raw = os.environ.get(SEARCH_ROOTS_ENV, "")
+    if not raw:
+        return []
+    return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
```

```
+
+
+def _basename(raw_path: str) -> str:
+    # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
+    win_name = PureWindowsPath(raw_path).name
+    return win_name or Path(raw_path).name
+
+
+def _candidate_paths(
+    raw_path: str,
+    field: str,
+    manifest_path: Path,
+    root: Path,
+    extra_roots: Sequence[Path],
+) -> list[Path]:
+    basename = _basename(raw_path)
+    raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
+    manifest_dir = manifest_path.resolve().parent
+    candidates: list[Path] = [raw]
+
+    if not raw.is_absolute():
+        candidates.extend([manifest_dir / raw, root / raw])
+
+    for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
+        candidates.append(directory / basename)
+    return _dedup_paths(candidates)
+
+
+def _candidate_dirs(
+    field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
+) -> list[Path]:
+    field_dir = _FIELD_DIR.get(field)
+    dirs: list[Path] = [manifest_dir]
+    if field_dir:
+        dirs.extend(
+            [
+                root.parent / "Annotation Setup" / field_dir,
+                root.parent / "Annotation Setup" / "Collect" / field_dir,
+                root / "output",
+            ]
+        )
+    dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
+    return _dedup_paths(dirs)
+
+
+def _expand_root(root: Path, field: str) -> list[Path]:
+    field_dir = _FIELD_DIR.get(field)
+    dirs = [root, root / "output"]
+    if field_dir:
+        dirs.extend(
+            [
+                root / field_dir,
+                root / "Annotation Setup" / field_dir,
+                root / "Annotation Setup" / "Collect" / field_dir,
+            ]
+        )
+    return dirs
+
+
+def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
+    out: list[Path] = []
+    seen: set[str] = set()
+    for path in paths:
+        try:
+            key = os.path.normcase(os.path.abspath(os.fspath(path)))
```

```
+        except OSError:
+            key = os.fspath(path)
+        if key in seen:
+            continue
+        seen.add(key)
+        out.append(path)
+    return out
+
+
+def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) -> None:
+    """Write a resolved manifest JSON atomically enough for local tooling."""
+
+    out = Path(path)
+    out.parent.mkdir(parents=True, exist_ok=True)
+    tmp = out.with_name(out.name + ".tmp")
+    with tmp.open("w", encoding="utf-8") as f:
+        json.dump(list(specs), f, indent=2)
+        f.write("\n")
+    os.replace(tmp, out)
+
+
+def build_parser() -> argparse.ArgumentParser:
+    p = argparse.ArgumentParser(
+        description="Check/rebase the local golden corpus manifest paths."
+    )
+    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
+    p.add_argument(
+        "--repo-root",
+        default=None,
+        help="repo root used for default sibling output/Annotation Setup lookups",
+    )
+    p.add_argument(
+        "--root",
+        action="append",
+        default=[],
+        help=(
+            "additional search root; repeatable. Also accepts roots via "
+            f"{SEARCH_ROOTS_ENV} separated by os.pathsep."
+        ),
+    )
+    p.add_argument(
+        "--require",
+        action="append",
+        choices=sorted(_FIELD_DIR),
+        default=None,
+        help="path field to require; repeatable. Defaults to trajectory and labels.",
+    )
+    p.add_argument(
+        "--min-count",
+        type=int,
+        default=None,
+        help="fail unless at least this many manifest rows are present",
+    )
+    p.add_argument(
+        "--write",
+        default=None,
+        help="optional path to write the resolved local manifest JSON",
+    )
+    return p
+
+
+def main(argv: Sequence[str] | None = None) -> int:
+    args = build_parser().parse_args(argv)
+    required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
+    try:
```

```
+        original = load_manifest(args.manifest)
+        resolved, issues = resolve_manifest_specs(
+            original,
+            args.manifest,
+            repo_root_path=args.repo_root,
+            extra_roots=args.root,
+            required_fields=required,
+        )
+    except (OSError, ValueError, json.JSONDecodeError) as e:
+        print(f"ERROR: {e}", file=sys.stderr)
+        return 2
+
+    if args.min_count is not None and len(resolved) < args.min_count:
+        issues.append(
+            ManifestIssue(
+                "manifest",
+                "rows",
+                str(len(resolved)),
+                f"expected at least {args.min_count} row(s)",
+            )
+        )
+
+    if issues:
+        print(
+            f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
+            file=sys.stderr,
+        )
+        for issue in issues:
+            print(f"  - {issue.message()}", file=sys.stderr)
+        return 2
+
+    rebased = sum(
+        1
+        for before, after in zip(original, resolved)
+        for field in required
+        if str(before.get(field, "")) != str(after.get(field, ""))
+    )
+    print(
+        f"OK: {args.manifest} has {len(resolved)} video(s); "
+        f"{len(resolved) * len(required)} required path(s) present; "
+        f"{rebased} path(s) rebased."
+    )
+    if args.write:
+        write_manifest(args.write, resolved)
+        print(f"wrote resolved manifest: {args.write}")
+    return 0
+
+
+if __name__ == "__main__":
+    raise SystemExit(main())
diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
index 0acca2d..3846ce0 100644
--- a/backend/eval/rally_seg_eval.py
+++ b/backend/eval/rally_seg_eval.py
@@ -27,6 +27,7 @@ import os
 from typing import Any, Dict, List, Optional, Tuple

 from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import Interval
 from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

@@ -185,8 +186,7 @@ def load_golden(
 ) -> List[Dict[str, Any]]:
     """Load each golden video's trajectory path + fps + frame_width (manifest) and GT rally
```

```
     intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
-    with open(manifest_path) as fh:
-        manifest = json.load(fh)
+    manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
     out: List[Dict[str, Any]] = []
     for v in manifest:
         feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
diff --git a/backend/eval/rally_seq_proto.py b/backend/eval/rally_seq_proto.py
index 2ea9483..b017f94 100644
--- a/backend/eval/rally_seq_proto.py
+++ b/backend/eval/rally_seq_proto.py
@@ -29,7 +29,6 @@ Run:
 from __future__ import annotations

 import argparse
-import json
 from typing import Dict, List, Tuple

 import numpy as np
@@ -38,6 +37,7 @@ from scipy.stats import rankdata

 from backend.eval.calibrate_wasb import load_golden_gts
 from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import score

 # Feature extraction is single-sourced in backend/features (ADR-008): training and
@@ -196,8 +196,7 @@ def main() -> None:
         help="JSON: [{name,trajectory,fps,frame_width,labels}, ...]",
     )
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        run(json.load(f))
+    run(load_resolved_manifest(args.manifest))


 if __name__ == "__main__":
diff --git a/backend/eval/serve_contrast.py b/backend/eval/serve_contrast.py
index 328857b..0f1b1f4 100644
--- a/backend/eval/serve_contrast.py
+++ b/backend/eval/serve_contrast.py
@@ -32,6 +32,7 @@ from typing import Any, Dict, List, Sequence, Tuple
 from backend.eval import windowing as W
 from backend.eval.calibrate_wasb import load_golden_gts
 from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import score
 from backend.eval.promotion import promote_if_served_safe
 from backend.pipeline.detectors import rally_gate
@@ -194,8 +195,7 @@ def main() -> None:
     )
     args = ap.parse_args()

-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     result = run_contrast(
         specs,
         iou=args.iou,
diff --git a/backend/eval/served_gate_a.py b/backend/eval/served_gate_a.py
index 994b9c1..c2151fb 100644
--- a/backend/eval/served_gate_a.py
+++ b/backend/eval/served_gate_a.py
@@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
```

```
 from __future__ import annotations

 import argparse
-import json
 from dataclasses import dataclass
 from typing import Any, Dict, List, Optional, Tuple

 from backend.config.models import FusionConfig, IndexingConfig, WasbIndexConfig
 from backend.eval import segmentation_metrics as _segm
 from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import Interval, Score, score
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
 from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
@@ -196,8 +196,7 @@ def format_table(
 def run(
     manifest_path: str, iou: float = 0.5
 ) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
     results = evaluate_manifest(specs, iou=iou)
     return results, aggregate(results)
```

12. assistant (2026-07-03T17:34:35.832Z)

Now let me check the tests to understand the expected behavior:

13. user (2026-07-03T17:34:36.669Z)

```
1       import json
2       from pathlib import Path
3
4       import pytest
5
6       from backend.eval import golden_manifest as gm
7
8
9       def _write(path: Path, body: str = "x\n") -> Path:
10          path.parent.mkdir(parents=True, exist_ok=True)
11          path.write_text(body, encoding="utf-8")
12          return path
13
14
15      def _write_manifest(repo: Path, rows: list[dict]) -> Path:
16          path = repo / "output" / "human_lovo_manifest.json"
17          path.parent.mkdir(parents=True, exist_ok=True)
18          path.write_text(json.dumps(rows), encoding="utf-8")
19          return path
20
21
22      def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23          workspace = tmp_path / "workspace"
24          repo = workspace / "badminton-highlight-indexer"
25          repo.mkdir(parents=True)
26          traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27          old_label = _write(
28              workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
29          )
30          output_label = _write(repo / "output" / "Bar.rallies.csv")
31          manifest = _write_manifest(
32              repo,
33              [
34                  {
35                      "name": "foo",
```

```
                    "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
                    "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
                    "fps": 30,
                    "frame_width": 1920,
                },
                {
                    "name": "bar",
                    "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
                    "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
                    "fps": 30,
                    "frame_width": 1920,
                },
            ],
        )

        specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)

        assert issues == []
        assert Path(specs[0]["trajectory"]) == traj.resolve()
        assert Path(specs[0]["labels"]) == old_label.resolve()
        assert Path(specs[1]["labels"]) == output_label.resolve()


    def test_strict_loader_reports_missing_required_files(tmp_path):
        repo = tmp_path / "repo"
        repo.mkdir()
        manifest = _write_manifest(
            repo,
            [
                {
                    "name": "missing",
                    "trajectory": "C:/old/missing_traj.csv",
                    "labels": "C:/old/missing.rallies.csv",
                }
            ],
        )

        with pytest.raises(gm.ManifestPathError) as excinfo:
            gm.load_resolved_manifest(manifest, repo_root_path=repo)

        assert len(excinfo.value.issues) == 2
        assert "file not found" in str(excinfo.value)


    def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
        workspace = tmp_path / "workspace"
        repo = workspace / "badminton-highlight-indexer"
        repo.mkdir(parents=True)
        traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
        manifest = _write_manifest(
            repo,
            [
                {
                    "name": "foo",
                    "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
                    "labels": "C:/old/missing.rallies.csv",
                }
            ],
        )

        specs, issues = gm.resolve_manifest_paths(
            manifest, repo_root_path=repo, required_fields=("trajectory",)
```

```
98          )
99
100         assert issues == []
```

14. user (2026-07-03T17:34:38.310Z)

```
diff --git a/tests/test_served_gate_a_regression.py b/tests/test_served_gate_a_regression.py
index 7fd7630..56848df 100644
--- a/tests/test_served_gate_a_regression.py
+++ b/tests/test_served_gate_a_regression.py
@@ -37,7 +37,6 @@ re-measurement as the corpus grows.

 from __future__ import annotations

-import json
 import os

 import pytest
@@ -49,6 +48,10 @@ pytest.importorskip("cv2")
 from backend.eval import segmentation_metrics as _segm  # noqa: E402
 from backend.eval import served_gate_a as sga  # noqa: E402
 from backend.eval.calibrate_wasb import load_golden_gts  # noqa: E402
+from backend.eval.golden_manifest import (  # noqa: E402
+    ManifestPathError,
+    load_resolved_manifest,
+)
 from backend.eval.metrics import score  # noqa: E402
 from backend.pipeline.detectors import rally_gate  # noqa: E402
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner  # noqa: E402
@@ -64,18 +67,14 @@ MANIFEST = os.environ.get(


 def _golden_specs():
-    """Return the manifest specs whose trajectory + label files are all present, else []."""
+    """Return rebased manifest specs when trajectory + label files are present, else []."""
     if not os.path.isfile(MANIFEST):
         return []
-    with open(MANIFEST, encoding="utf-8") as f:
-        specs = json.load(f)
-    ok = [
-        s
-        for s in specs
-        if os.path.isfile(str(s.get("trajectory", "")))
-        and os.path.isfile(str(s.get("labels", "")))
-    ]
-    return ok if len(ok) >= 6 else []
+    try:
+        specs = load_resolved_manifest(MANIFEST)
+    except ManifestPathError:
+        return []
+    return specs if len(specs) >= 6 else []


 _SPECS = _golden_specs()
```

15. assistant (2026-07-03T17:34:40.275Z)

Now let me check the key concern areas - the workspace layout hardcoding and logging:

16. user (2026-07-03T17:34:41.607Z)

```
235:                root.parent / "Annotation Setup" / field_dir,
236:                root.parent / "Annotation Setup" / "Collect" / field_dir,
```

17. user (2026-07-03T17:34:42.711Z)

```
 from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
     leave_one_video_out,
     select_best,
 )
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.pipeline.detectors import rally_gate
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
     ap.add_argument("--iou", type=float, default=0.5)
     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, metric=args.metric)


diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--

 from backend.eval import windowing as _windowing
 from backend.eval.calibrate_wasb import load_golden_gts
 from backend.eval.classifier import LogisticRegression
 from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import score
 from backend.eval.training import label_candidates
 from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
     )
     ap.add_argument("--model-out", default="output/local_rally_model.json")
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)


diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index dd3f6a1..aee6d7e 100644
--
+++ b/backend/eval/fusion_golden.py
@@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional

 from backend.eval import distill_local
 from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.training import label_candidates
 from backend.pipeline.detectors import fusion_features as ff
 from backend.pipeline.detectors import rally_gate
@@ -160,8 +161,7 @@ def run(
     fresh: bool = False,
     smallest_first: bool = False,
 ) -> List[str]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
     if smallest_first:
         # Process small videos first for quick feedback (the slow full match lands last);
         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
```

```
row/frame).
diff --git a/backend/eval/golden_manifest.py b/backend/eval/golden_manifest.py
new file mode 100644
--
+        extra_roots=extra_roots,
+        required_fields=required_fields,
+    )
+
+
+def load_resolved_manifest(
+    manifest_path: str | os.PathLike[str],
+    *,
+    repo_root_path: str | os.PathLike[str] | None = None,
+    extra_roots: Iterable[str | os.PathLike[str]] = (),
+    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
--
+++ b/backend/eval/rally_seg_eval.py
@@ -27,6 +27,7 @@ import os
```

18. assistant (2026-07-03T17:34:45.208Z)

Let me look at how the output message is used and if there's any diagnostic capability:

19. user (2026-07-03T17:34:46.168Z)

```
325     def main(argv: Sequence[str] | None = None) -> int:
326         args = build_parser().parse_args(argv)
327         required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
328         try:
329             original = load_manifest(args.manifest)
330             resolved, issues = resolve_manifest_specs(
331                 original,
332                 args.manifest,
333                 repo_root_path=args.repo_root,
334                 extra_roots=args.root,
335                 required_fields=required,
336             )
337         except (OSError, ValueError, json.JSONDecodeError) as e:
338             print(f"ERROR: {e}", file=sys.stderr)
339             return 2
340
341         if args.min_count is not None and len(resolved) < args.min_count:
342             issues.append(
343                 ManifestIssue(
344                     "manifest",
345                     "rows",
346                     str(len(resolved)),
347                     f"expected at least {args.min_count} row(s)",
348                 )
349             )
350
351         if issues:
352             print(
353                 f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
354                 file=sys.stderr,
355             )
356             for issue in issues:
357                 print(f"  - {issue.message()}", file=sys.stderr)
358             return 2
359
360         rebased = sum(
361             1
362             for before, after in zip(original, resolved)
363             for field in required
364             if str(before.get(field, "")) != str(after.get(field, ""))
```

```
365         )
366         print(
367             f"OK: {args.manifest} has {len(resolved)} video(s); "
368             f"{len(resolved) * len(required)} required path(s) present; "
369             f"{rebased} path(s) rebased."
370         )
371         if args.write:
372             write_manifest(args.write, resolved)
373             print(f"wrote resolved manifest: {args.write}")
374         return 0
375
376
377     if __name__ == "__main__":
378         raise SystemExit(main())
379
```

20. user (2026-07-03T17:34:47.601Z)

```
The corpus is now large enough to move from "grow labels first" into alpha-readiness work, but
the 15-video headline is not itself a launch gate pass. `[verified]` The golden manifest has 15
badminton videos; `[analysis]` the local manifest needed path rebasing before manifest-driven
commands ran; `[analysis]` only 6 of 15 feature JSONs had the full fusion schema; `[analysis]`
the n=15 quality baseline is stale and must be intentionally refreshed before it can gate
anything.
- `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md` says `output/human_lovo_manifest.json` is
the ingested golden corpus at `n = 15`.
- `[analysis]` The local manifest had stale absolute paths. A temporary rebased manifest found
all 15 trajectories and labels in this workspace.
- `[analysis]` `python -m backend.eval.serve_contrast --manifest <rebased-n15>` measured
proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.
- `[analysis]` `python -m training.gen0.harness --manifest <rebased-n15> --floor
eval_baselines/heuristic_lovo_n6.json --version analysis-n15 --out <temp-out>` produced learned
LOVO mean F1 `0.551`, `ship=False`; blockers remain stale n=6 floor/sign test, uncalibrated ship
target, and WASB C3 provenance.
- `[analysis]` `python scripts/nightly_regression.py --manifest <rebased-n15> --features-dir
output --no-report` reported default/milestone tolF1 `0.3636` vs baseline-off tolF1 `0.1382`,
but marked the frozen baseline stale due to config/corpus changes.
| Stale local manifests create false confidence | Manifest-driven tools skipped or failed until
paths were rebased | Make the corpus manifest portable and add a path/fixture smoke check |
| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that
fails loudly when labels or trajectories are missing | `badminton-highlight-indexer` | A0 | No |
IN PROGRESS | this PR |
| A3 | Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile
collector's six-video source manifest with the 15-video eval corpus | `sports-data-collector` +
vault | A2 | Owner upload/storage scope | TODO | - |
2. **Corpus portability PR:** rebase/portable manifest tooling plus missing-file smoke test.
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
--write output/human_lovo_manifest.local.json
4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a checked-
in redacted manifest, or both?
- [ ] n=15 corpus is portable enough that manifest-driven quality commands do not rely on stale
local absolute paths.
python -m backend.eval.serve_contrast --manifest <rebased-n15-manifest>
python -m training.gen0.harness --manifest <rebased-n15-manifest> --floor
eval_baselines\heuristic_lovo_n6.json --version analysis-n15 --out <temp-out>
python scripts\nightly_regression.py --manifest <rebased-n15-manifest> --features-dir output
--no-report
```

21. assistant (2026-07-03T17:34:50.960Z)

Let me check if there's any data about which files were matched/rebased in the library callers:

22. user (2026-07-03T17:34:51.828Z)

```
110     def load_resolved_manifest(
111         manifest_path: str | os.PathLike[str],
112         *,
113         repo_root_path: str | os.PathLike[str] | None = None,
114         extra_roots: Iterable[str | os.PathLike[str]] = (),
115         required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
116         strict: bool = True,
117     ) -> list[dict[str, Any]]:
118         """Load a manifest with stale paths rebased; optionally raise on missing files."""
119 
120         specs, issues = resolve_manifest_paths(
121             manifest_path,
122             repo_root_path=repo_root_path,
123             extra_roots=extra_roots,
124             required_fields=required_fields,
125         )
126         if strict and issues:
127             raise ManifestPathError(manifest_path, issues)
128         return specs
129 
130 
131     def resolve_manifest_specs(
132         specs: Sequence[dict[str, Any]],
133         manifest_path: str | os.PathLike[str],
134         *,
135         repo_root_path: str | os.PathLike[str] | None = None,
136         extra_roots: Iterable[str | os.PathLike[str]] = (),
137         required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
138     ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
139         """Resolve an already-loaded manifest list."""
```

23. user (2026-07-03T17:34:53.016Z)

```
5a91701 eval: add portable golden manifest check
```

24. assistant (2026-07-03T17:34:55.392Z)

```
Let me check further if library callers have access to diagnostics and look at the documentation
on Q4:
```

25. user (2026-07-03T17:34:56.852Z)

```
(Bash completed with no output)
```

26. user (2026-07-03T17:34:59.818Z)

9. Open questions

```
1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should
P10/P11 be reopened before any tester wave?
2. **Owner/platform:** Which hosted compute target is the next stand-up: Cloud Run GPU, GCE L4
VM, or another approved GPU host?
3. **Owner/legal:** What is the acceptable alpha wording while WASB C3 remains unresolved?
4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a checked-
in redacted manifest, or both?
5. **Engineering:** Should n=15 feature regeneration target full fusion schema for every clip,
or should the eval loaders support mixed schemas with explicit capability tags?
```

10. Definition of done

```
- [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without SSH.
```

27. assistant (2026-07-03T17:35:01.788Z)

Now let me check if there's any logging/tracking of which paths were resolved:

28. user (2026-07-03T17:35:04.209Z)

```
load_resolved_manifest signature:
(manifest_path: 'str | os.PathLike[str]', *, repo_root_path: 'str | os.PathLike[str] | None' =
None, extra_roots: 'Iterable[str | os.PathLike[str]]' = (), required_fields: 'Sequence[str]' =
('trajectory', 'labels'), strict: 'bool' = True) -> 'list[dict[str, Any]]'

resolve_manifest_specs signature:
(specs: 'Sequence[dict[str, Any]]', manifest_path: 'str | os.PathLike[str]', *, repo_root_path:
'str | os.PathLike[str] | None' = None, extra_roots: 'Iterable[str | os.PathLike[str]]' = (),
required_fields: 'Sequence[str]' = ('trajectory', 'labels')) -> 'tuple[list[dict[str, Any]],
list[ManifestIssue]]'
```

29. assistant (2026-07-03T17:35:06.945Z)

Let me check where training and A3 work would need path resolution:

30. user (2026-07-03T17:35:08.506Z)

```
./backend/eval/batch.py
./backend/eval/calibrate_local.py
./backend/eval/distill_local.py
./backend/eval/fusion_audio.py
./backend/eval/fusion_golden.py
./backend/eval/golden_fixtures.py
./backend/eval/golden_manifest.py
./backend/eval/golden_real_fixtures.py
./backend/eval/rally_seg_eval.py
./backend/eval/rally_seq_proto.py
./backend/eval/served_gate_a.py
./backend/eval/serve_contrast.py
./backend/eval/splits.py
./backend/features/rally_seq.py
./backend/flywheel.py
./backend/pipeline/detectors/wasb_infer.py
./backend/worker/__main__.py
```

31. user (2026-07-03T17:35:11.512Z)

```
diff --git a/training/gen0/harness.py b/training/gen0/harness.py
index 6eec29f..433d2cb 100644
--- a/training/gen0/harness.py
+++ b/training/gen0/harness.py
@@ -36,6 +36,7 @@ from typing import Dict, List, Optional
 import numpy as np

 from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.gt_loader import load_gt_intervals
 from backend.eval.rally_seq_proto import (
     _seg_f1,
@@ -217,8 +218,7 @@ def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
 def run(
     manifest_path: str, out_dir: str, version: str, floor_path: Optional[str] = None
 ) -> dict:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
     floor = None
     if floor_path:
         with open(floor_path, encoding="utf-8") as f:
```

32. assistant (2026-07-03T17:35:15.518Z)

Now let me understand the critical point about logging and diagnostics when library code calls
the resolver:

33. user (2026-07-03T17:35:16.215Z)

```
131     def resolve_manifest_specs(
132         specs: Sequence[dict[str, Any]],
133         manifest_path: str | os.PathLike[str],
134         *,
135         repo_root_path: str | os.PathLike[str] | None = None,
136         extra_roots: Iterable[str | os.PathLike[str]] = (),
137         required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
138     ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
139         """Resolve an already-loaded manifest list."""
140 
141         manifest = Path(manifest_path)
142         root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
143         roots = [Path(p) for p in _env_roots()]
144         roots.extend(Path(p) for p in extra_roots)
145         required = tuple(required_fields)
146 
147         resolved: list[dict[str, Any]] = []
148         issues: list[ManifestIssue] = []
149         for idx, spec in enumerate(specs):
150             row = dict(spec)
151             name = str(row.get("name") or f"row[{idx}]")
152             for field in required:
153                 raw = row.get(field)
154                 if not raw:
155                     issues.append(ManifestIssue(name, field, "", "required field missing"))
156                     continue
157                 hit = resolve_existing_path(
158                     str(raw),
159                     field,
160                     manifest_path=manifest,
161                     repo_root_path=root,
162                     extra_roots=roots,
163                 )
164                 if hit is None:
165                     issues.append(ManifestIssue(name, field, str(raw), "file not found"))
166                 else:
167                     row[field] = str(hit)
168             resolved.append(row)
169         return resolved, issues
170 
171 
172     def resolve_existing_path(
173         raw_path: str,
174         field: str,
175         *,
176         manifest_path: str | os.PathLike[str],
177         repo_root_path: str | os.PathLike[str],
178         extra_roots: Iterable[str | os.PathLike[str]] = (),
179     ) -> Path | None:
180         """Resolve one manifest path field to an existing file, if possible."""
```

34. user (2026-07-03T17:35:19.962Z)

Resolved spec keys: dict_keys(['name', 'trajectory', 'labels'])
Library caller sees no diagnostic info about which paths were rebased!

35. assistant (2026-07-03T17:35:23.841Z)

Perfect. Now let me check the actual implementation of basename-only matching to understand the collision risk:

36. user (2026-07-03T17:35:24.816Z)

```
200
201     def _basename(raw_path: str) -> str:
202         # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
203         win_name = PureWindowsPath(raw_path).name
204         return win_name or Path(raw_path).name
205
206
207     def _candidate_paths(
208         raw_path: str,
209         field: str,
210         manifest_path: Path,
211         root: Path,
212         extra_roots: Sequence[Path],
213     ) -> list[Path]:
214         basename = _basename(raw_path)
215         raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216         manifest_dir = manifest_path.resolve().parent
217         candidates: list[Path] = [raw]
218
219         if not raw.is_absolute():
220             candidates.extend([manifest_dir / raw, root / raw])
221
222         for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223             candidates.append(directory / basename)
224         return _dedup_paths(candidates)
225
226
227     def _candidate_dirs(
228         field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
229     ) -> list[Path]:
230         field_dir = _FIELD_DIR.get(field)
231         dirs: list[Path] = [manifest_dir]
232         if field_dir:
233             dirs.extend(
234                 [
235                     root.parent / "Annotation Setup" / field_dir,
236                     root.parent / "Annotation Setup" / "Collect" / field_dir,
237                     root / "output",
238                 ]
239             )
240         dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241         return _dedup_paths(dirs)
242
243
244     def _expand_root(root: Path, field: str) -> list[Path]:
245         field_dir = _FIELD_DIR.get(field)
246         dirs = [root, root / "output"]
247         if field_dir:
248             dirs.extend(
249                 [
250                     root / field_dir,
251                     root / "Annotation Setup" / field_dir,
252                     root / "Annotation Setup" / "Collect" / field_dir,
253                 ]
254             )
255         return dirs
256
257
258     def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
259         out: list[Path] = []
```

```
260         seen: set[str] = set()
261         for path in paths:
262             try:
263                 key = os.path.normcase(os.path.abspath(os.fspath(path)))
264             except OSError:
265                 key = os.fspath(path)
266             if key in seen:
267                 continue
268             seen.add(key)
269             out.append(path)
```

37. assistant (2026-07-03T17:35:27.746Z)

Now let me verify the order in which candidates are checked:

38. user (2026-07-03T17:35:27.955Z)

```
172     def resolve_existing_path(
173         raw_path: str,
174         field: str,
175         *,
176         manifest_path: str | os.PathLike[str],
177         repo_root_path: str | os.PathLike[str],
178         extra_roots: Iterable[str | os.PathLike[str]] = (),
179     ) -> Path | None:
180         """Resolve one manifest path field to an existing file, if possible."""
181 
182         for candidate in _candidate_paths(
183             raw_path,
184             field,
185             Path(manifest_path),
186             Path(repo_root_path),
187             [Path(p) for p in extra_roots],
188         ):
189             if candidate.is_file():
190                 return candidate.resolve()
191         return None
```

39. assistant (2026-07-03T17:35:30.433Z)

Now let me check the documentation comment at the top and understand the intent of the fallback:

40. user (2026-07-03T17:35:30.770Z)

```
1       """Portable golden-corpus manifest loader and smoke checker.
2 
3       The owner-side golden manifest is intentionally gitignored and has historically
4       carried absolute paths from whichever box ingested a clip. This module keeps that
5       artifact usable after the workspace moves by resolving stale paths against the
6       current repo layout and failing loudly when required corpus files are absent.
7 
8       Default local layout supported:
9         - ../Annotation Setup/Trajectories/<trajectory>.csv
10        - ../Annotation Setup/Golden Labelled/<labels>.csv
11        - output/<labels>.csv
12 
13      Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14      """
15 
16      from __future__ import annotations
17 
18      import argparse
19      import json
20      import os
```

```
21      import sys
22      from dataclasses import dataclass
23      from pathlib import Path, PureWindowsPath
24      from typing import Any, Iterable, Sequence
25
26      DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27      SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28      DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30      _FIELD_DIR = {
31          "trajectory": "Trajectories",
32          "labels": "Golden Labelled",
33      }
34
35
```

41. assistant (2026-07-03T17:35:33.061Z)

Now let me check if there's any hashing/validation of resolved files:

42. user (2026-07-03T17:35:34.500Z)

```
    """Load the flat golden manifest list and validate its outer shape."""
```

43. assistant (2026-07-03T17:35:36.935Z)

Perfect. Now let me examine the tests to see if there are any tests for the collision/fallback scenario:

44. user (2026-07-03T17:35:37.091Z)

```
1       import json
2       from pathlib import Path
3
4       import pytest
5
6       from backend.eval import golden_manifest as gm
7
8
9       def _write(path: Path, body: str = "x\n") -> Path:
10          path.parent.mkdir(parents=True, exist_ok=True)
11          path.write_text(body, encoding="utf-8")
12          return path
13
14
15      def _write_manifest(repo: Path, rows: list[dict]) -> Path:
16          path = repo / "output" / "human_lovo_manifest.json"
17          path.parent.mkdir(parents=True, exist_ok=True)
18          path.write_text(json.dumps(rows), encoding="utf-8")
19          return path
20
21
22      def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23          workspace = tmp_path / "workspace"
24          repo = workspace / "badminton-highlight-indexer"
25          repo.mkdir(parents=True)
26          traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27          old_label = _write(
28              workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
29          )
30          output_label = _write(repo / "output" / "Bar.rallies.csv")
31          manifest = _write_manifest(
32              repo,
33              [
```

```
                {
                    "name": "foo",
                    "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
                    "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
                    "fps": 30,
                    "frame_width": 1920,
                },
                {
                    "name": "bar",
                    "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
                    "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
                    "fps": 30,
                    "frame_width": 1920,
                },
            ],
        )

        specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)

        assert issues == []
        assert Path(specs[0]["trajectory"]) == traj.resolve()
        assert Path(specs[0]["labels"]) == old_label.resolve()
        assert Path(specs[1]["labels"]) == output_label.resolve()


    def test_strict_loader_reports_missing_required_files(tmp_path):
        repo = tmp_path / "repo"
        repo.mkdir()
        manifest = _write_manifest(
            repo,
            [
                {
                    "name": "missing",
                    "trajectory": "C:/old/missing_traj.csv",
                    "labels": "C:/old/missing.rallies.csv",
                }
            ],
        )

        with pytest.raises(gm.ManifestPathError) as excinfo:
            gm.load_resolved_manifest(manifest, repo_root_path=repo)

        assert len(excinfo.value.issues) == 2
        assert "file not found" in str(excinfo.value)


    def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
        workspace = tmp_path / "workspace"
        repo = workspace / "badminton-highlight-indexer"
        repo.mkdir(parents=True)
        traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
        manifest = _write_manifest(
            repo,
            [
                {
                    "name": "foo",
                    "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
                    "labels": "C:/old/missing.rallies.csv",
                }
            ],
        )

```

```
        specs, issues = gm.resolve_manifest_paths(
            manifest, repo_root_path=repo, required_fields=("trajectory",)
        )

        assert issues == []
        assert Path(specs[0]["trajectory"]) == traj.resolve()
        assert specs[0]["labels"] == "C:/old/missing.rallies.csv"


    def test_cli_writes_resolved_manifest(tmp_path, capsys):
        workspace = tmp_path / "workspace"
        repo = workspace / "badminton-highlight-indexer"
        repo.mkdir(parents=True)
        _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
        _write(repo / "output" / "Foo.rallies.csv")
        manifest = _write_manifest(
            repo,
            [
                {
                    "name": "foo",
                    "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
                    "labels": "C:/old/badminton-highlight-indexer/output/Foo.rallies.csv",
                }
            ],
        )
        out = repo / "output" / "human_lovo_manifest.local.json"

        rc = gm.main(
            [
                "--manifest",
                str(manifest),
                "--repo-root",
                str(repo),
                "--min-count",
                "1",
                "--write",
                str(out),
            ]
        )

        captured = capsys.readouterr()
        assert rc == 0
        assert "OK:" in captured.out
        assert out.is_file()
        resolved = json.loads(out.read_text(encoding="utf-8"))
        assert Path(resolved[0]["trajectory"]).is_file()
        assert Path(resolved[0]["labels"]).is_file()


    def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
        repo = tmp_path / "repo"
        repo.mkdir()
        manifest = _write_manifest(
            repo,
            [{"name": "foo", "trajectory": "missing.csv", "labels": "missing.rallies.csv"}],
        )

        rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])

        captured = capsys.readouterr()
        assert rc == 2
        assert "golden manifest path check failed" in captured.err
        assert "foo: trajectory" in captured.err

```

45. assistant (2026-07-03T17:35:39.506Z)

Excellent. Now let me check where else manifest path resolution is needed (A3, A7):

46. user (2026-07-03T17:35:51.581Z)

```
-rw-r--r-- 1 avidu 197609  8102 Jun 29 13:47 flywheel.py
./backend/eval/annotations.py
./backend/eval/batch.py
./backend/eval/calibrate_wasb.py
./backend/eval/gt_loader.py
./backend/eval/splits.py
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
find: ?grep? terminated by signal 13
```

47. assistant (2026-07-03T17:36:07.846Z)

Now, let me compile my findings into a JSON analysis. Based on my thorough review:

ALTITUDE ISSUES FOUND:

1. **Line 223 & 235-236**: Basename-only fallback matching in `_candidate_dirs` ? adds `directory / basename` to search paths without any validation that the matched file is correct. If two videos have the same filename (even in different subdirectories), a wrong file would be silently accepted as golden data for corpus integrity.

2. **Line 110-128**: `load_resolved_manifest()` library function discards diagnostic information. It calls `resolve_manifest_paths()` which returns issues, but `load_resolved_manifest()` only exposes specs in strict mode. Library callers (distill_local, fusion_golden, serve_contrast, served_gate_a, rally_seg_eval, rally_seq_proto, calibrate_local) get no signal about which paths were rebased vs. original.

3. **Line 235-236 & 244-255**: Hardcoded workspace layout (`root.parent / "Annotation Setup"` + "Collect" legacy variant) bakes one owner's folder structure into repo code. The PR introduces path rebasing as mitigation, but the deeper fix would be relative paths at ingest time (producer-side fix in ingestion pipeline, not consumer-side workaround).

4. **Missing**: No validation of resolved files (row count sanity check, FPS check, hash comparison, or even logging which path was selected from candidates). A wrong same-named file passes silently.

5. **Line 70-83**: Manifest remains gitignored owner-side state. ALPHA_LAUNCH_READINESS.md Q4 asks "Should the portable corpus manifest live as a generated artifact, a checked-in redacted manifest, or both?" ? PR addresses portability/rebase but doesn't generate or check in the manifest, leaving it as ephemeral local state.

6. **File placement**: `backend/eval/` is specific to eval ? A3 (collector reconciliation) and A7 (feature regen) will also need path resolution. Should the module live in `backend/utils/` or `backend/data_pipeline/` for broader reuse, or should path resolution be a concern of only eval consumers?

```json
[
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 223,
```

```
    "summary": "Basename-only fallback allows silent collision: if two corpus videos share a
filename, wrong file accepted as golden data without any validation",
    "failure_scenario": "Video 'test.csv' exists in both output/ and Trajectories/; manifest
points to a stale path; resolver silently picks first match by basename, potentially using wrong
video for quality gates"
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 110,
    "summary": "load_resolved_manifest() discards rebase diagnostics; library callers get no
signal which paths were rebased vs. original",
    "failure_scenario": "distill_local.py calls load_resolved_manifest() and silently uses
rebased paths; operator has no log of path changes, making it hard to debug if wrong file was
matched"
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 235,
    "summary": "Hardcoded workspace layout (root.parent/'Annotation Setup'/'Collect') bakes
owner's folder structure into repo code; deeper fix is relative paths at ingest time, not
rebase-on-every-load",
    "failure_scenario": "Owner reorganizes workspace layout (renames Annotation Setup folder);
all manifest-driven tools fail until PR is reverted and ingest pipeline is fixed to write
relative paths"
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 172,
    "summary": "Resolved files are not validated (row count, fps, hash sanity); only file-exists
check performed",
    "failure_scenario": "Basename match picks a corrupted or wrong-format corpus file; resolver
accepts it; quality eval runs on wrong data, producing false-positive quality gates"
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 1,
    "summary": "Manifest remains gitignored owner-side state; PR adds portability but doesn't
address ALPHA_LAUNCH_READINESS Q4: should manifest be generated artifact or checked-in redacted
form?",
    "failure_scenario": "After alpha launch, no checked-in manifest artifact exists; new team
member clones repo, manifest is stale/missing, must rebase manually; no reproducibility
guarantee without checked-in portable manifest"
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 1,
    "summary": "Path resolution module in backend/eval/ is specific to eval; A3 (collector
reconciliation) and A7 (feature regen) will need same capability but module is not in shared
backend/utils or data_pipeline location",
    "failure_scenario": "A3/A7 implementation must either duplicate path-resolution logic or add
dependencies into backend/eval, violating layering; module should live in backend/utils or
data_pipeline for cross-workstream reuse"
  }
]
```